

gRASP A Manual

<https://zhaoli2042.github.io/gRASP A-mkdoc/>

None

None

Table of contents

1. Homepage	17
-------------	----

1.1



Capabilities	17
--------------	----

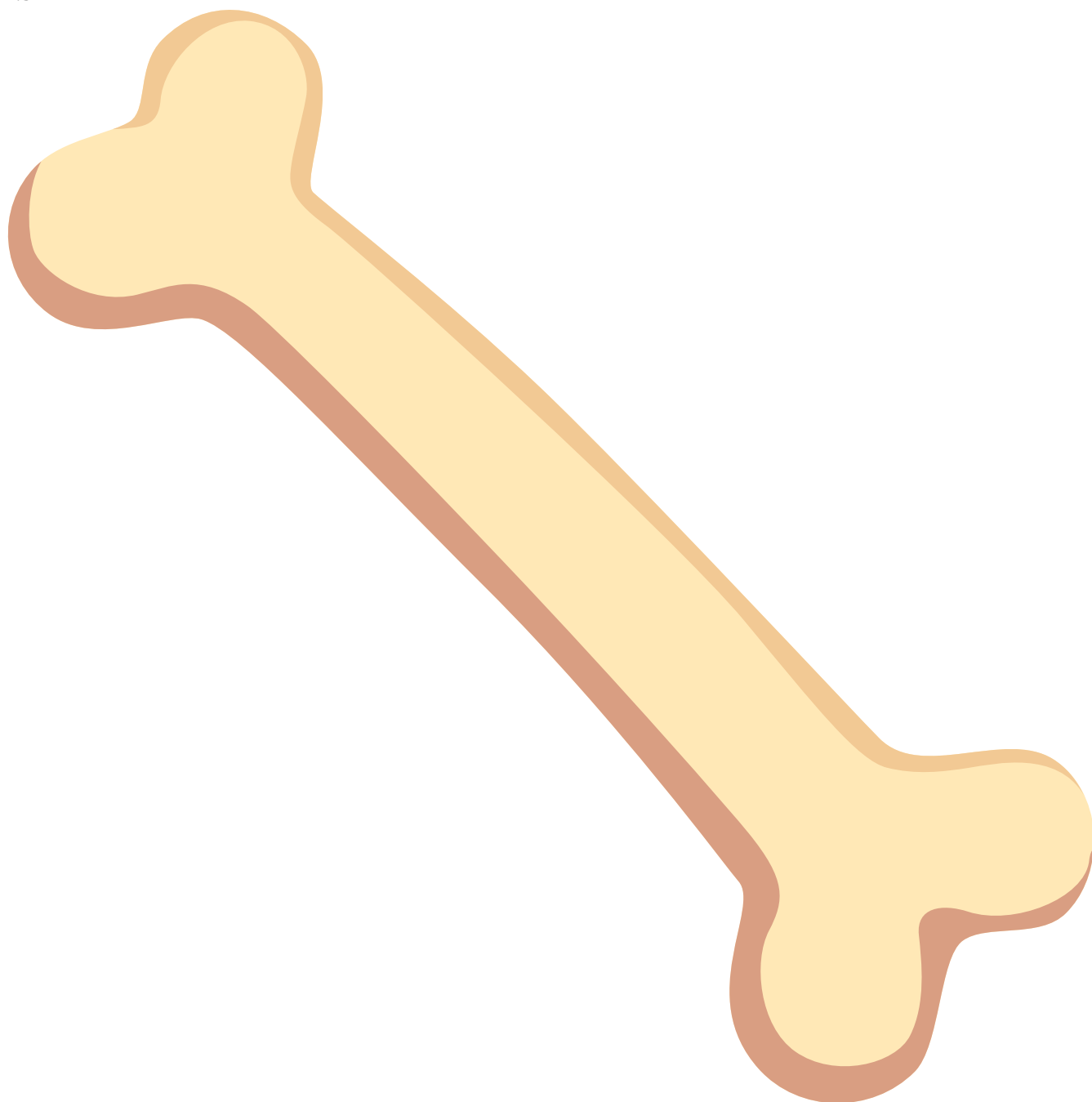
1.2



Moves

17

1.3



Backbones & Algorithms

17

2. Examples

18

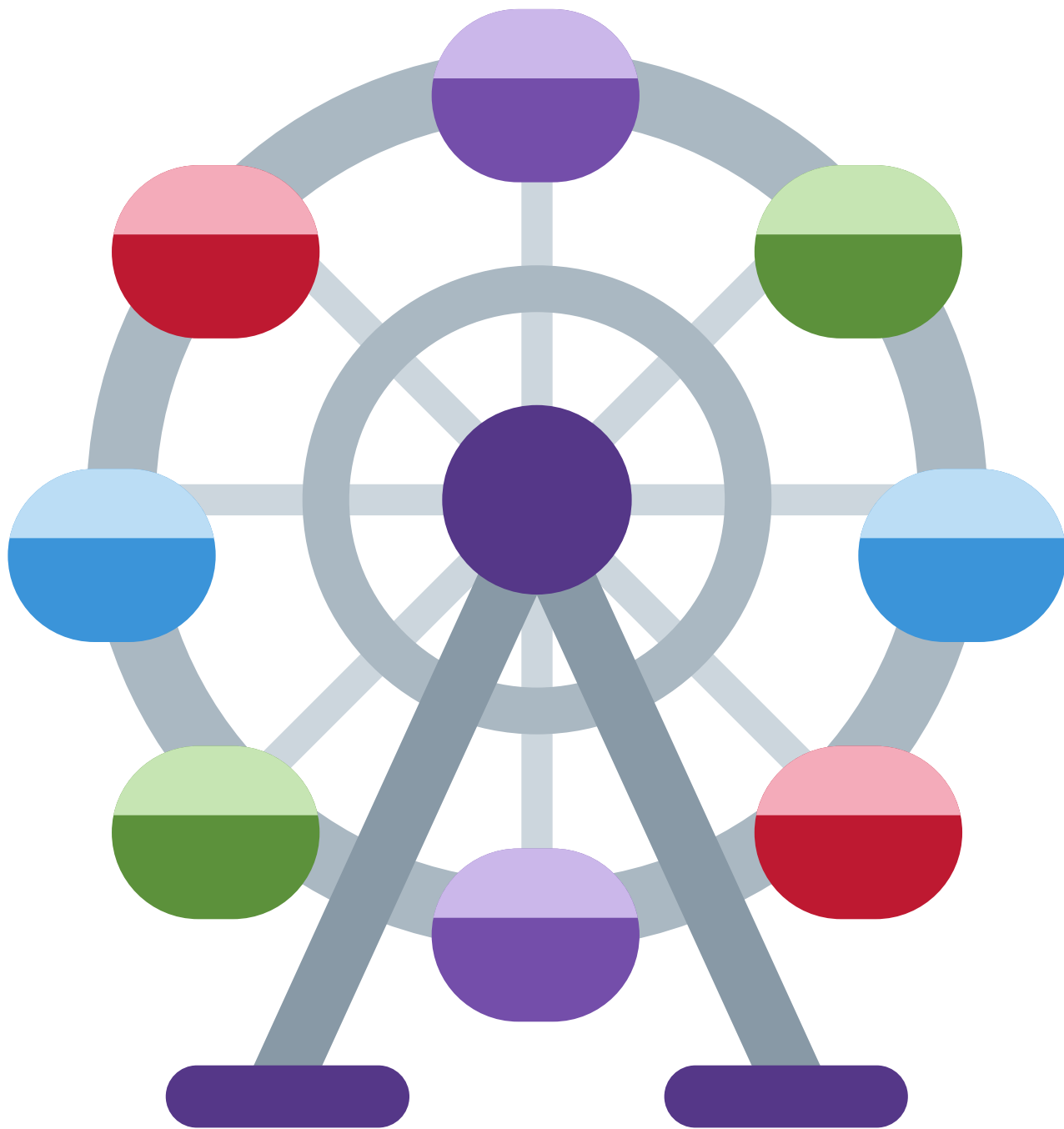
2.1



Basic GCMC simulation

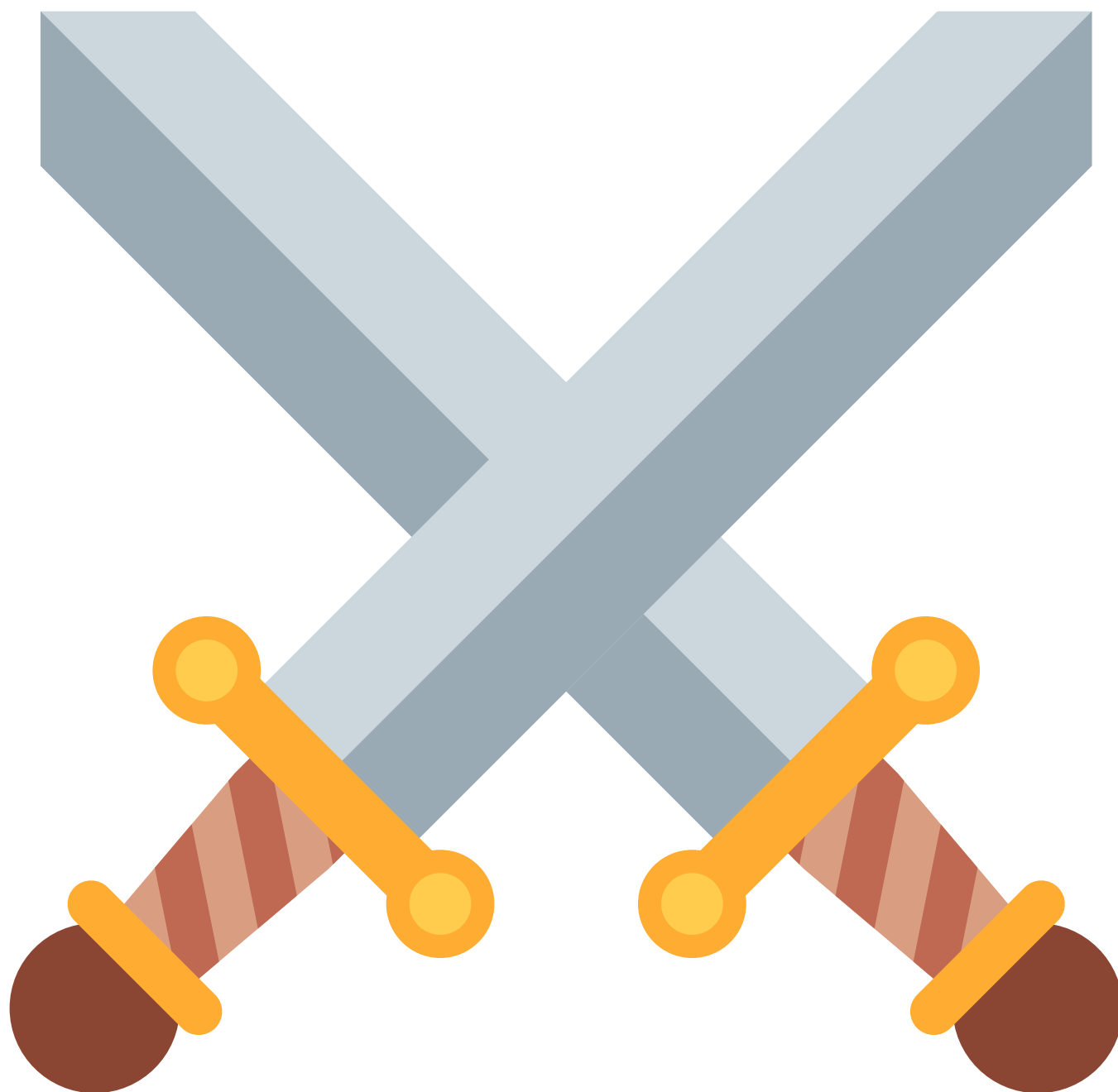
18

2.2



NU-2000 Linker Rotation

2.3



Energy Comparison: gRASP vs RASP-2

18

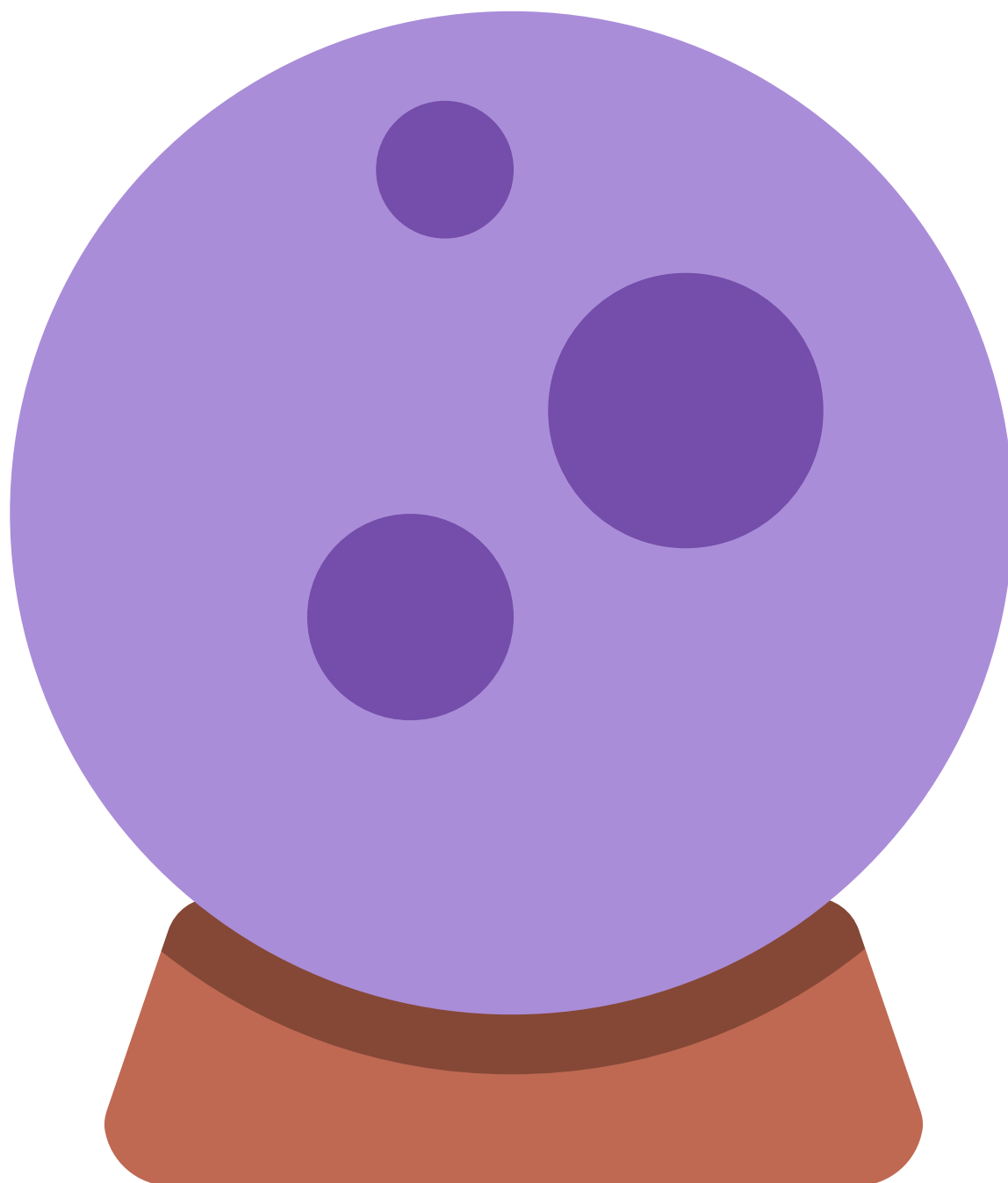
2.4



Mixture GCMC of CO₂/Methane

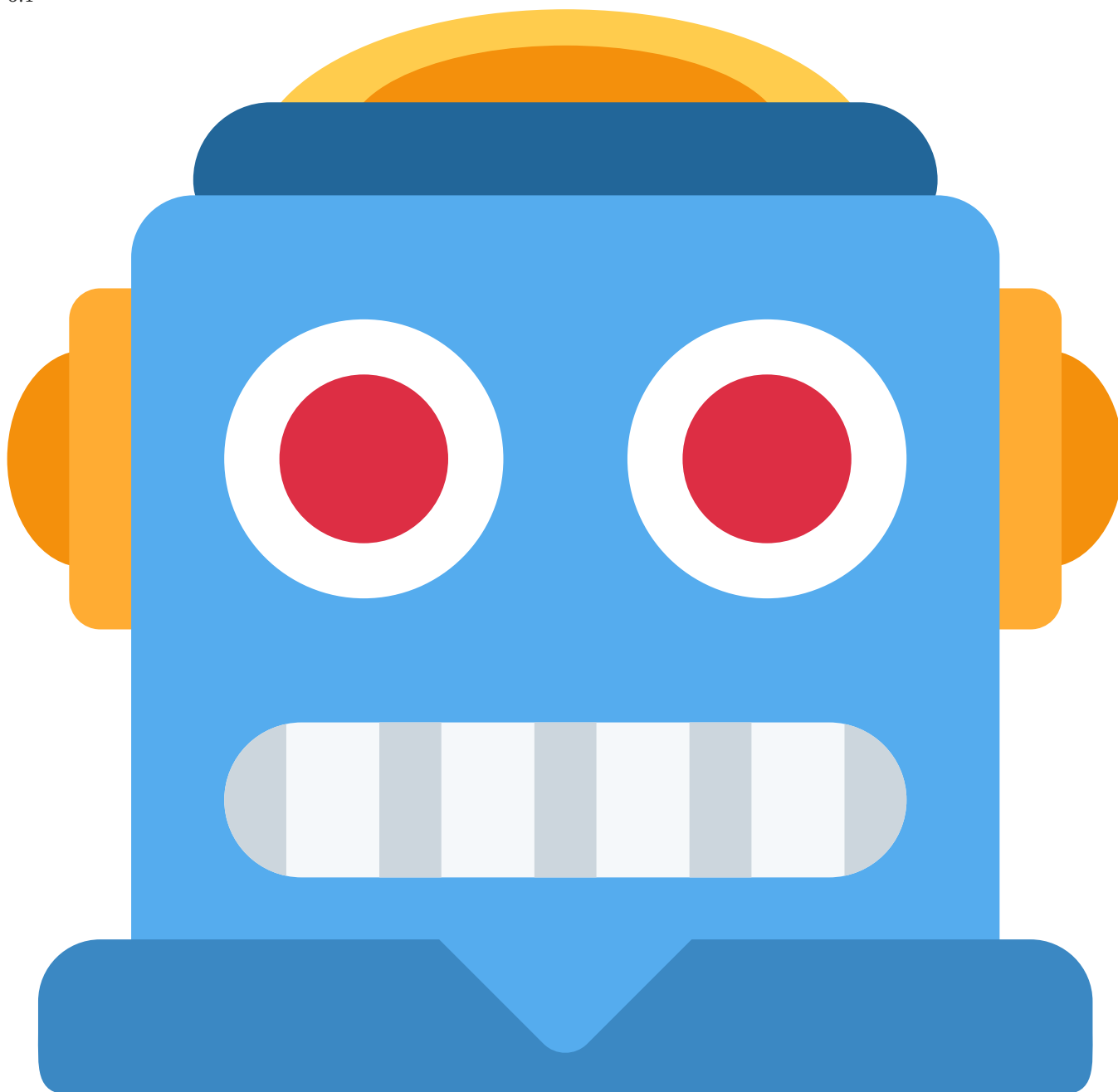
18

2.5



ML potential for framework-adsorbate interactions	18
3. Input File	19
3.1 Basics	19
3.2 simulation.input	20
3.3 force_field_mixing_rules.def	20
3.4 pseudo_atoms.def	20
3.5 CIF File	20
4. Installation	21
4.1 Prerequisites	21

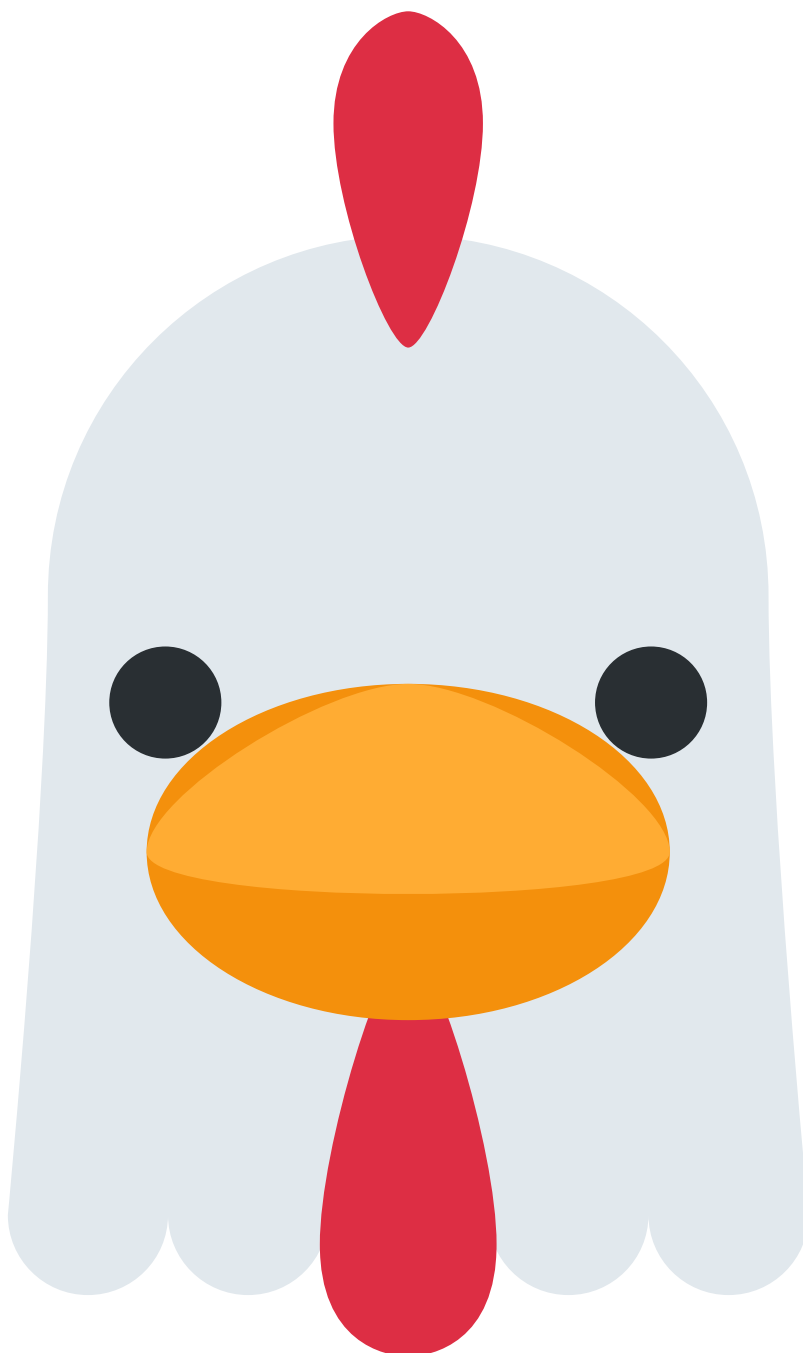
4.2 Steps (If you start with a clean Ubuntu 24.04)	21
5. Installation	23
6. Quick Start	24
6.1	



Installation	24
7. Required Keywords	26
7.1 Here are the required keywords in simulation.input	26



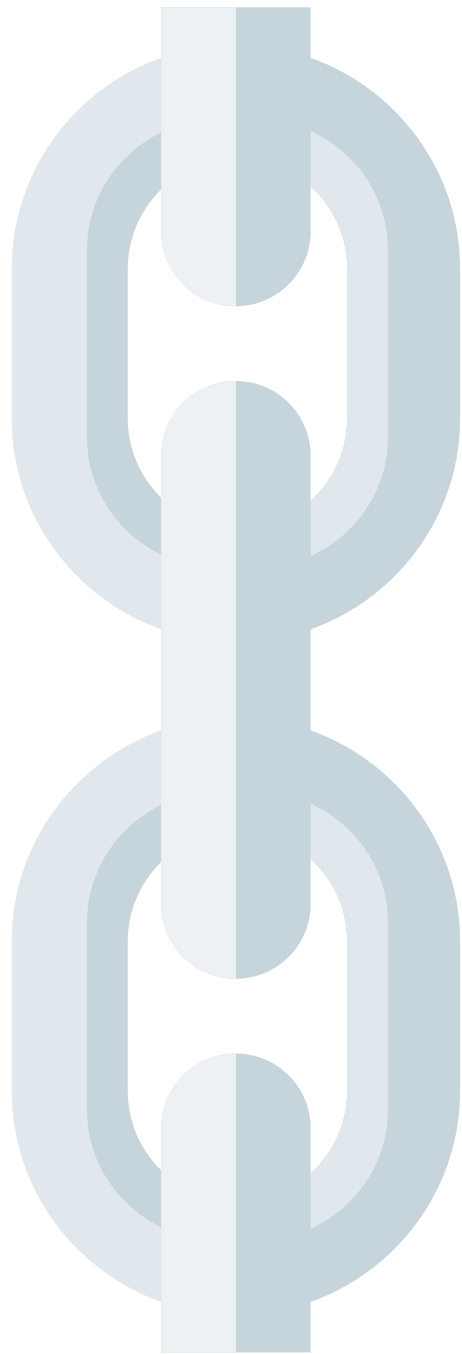
7.3



General Setup

26

7.4



7.5



7.6



Framework Setup

28

7.7



Adsorbate Setup	29
8. Chinese	30
8.1 主页	30
8.2 示例	31
8.3 输入文件	32
8.4 安装	34
8.5 快速开始	35
8.6 Required Keywords	37
8.7 Examples	41

1. Homepage

For the gRASPA code, visit [our Github page](#).

1.1 🦾 Capabilities

- **Grand Canonical Monte Carlo (GCMC)** simulations for small molecules (e.g. water/CO2/methane/...)
- **Transition-Matrix Monte Carlo** (TMMC) in the grand-canonical ensemble (GC-TMMC)
- **Mixture** adsorption Monte Carlo in the grand-canonical ensemble
- **NVT-Gibbs** Monte Carlo simulation (Available in Fast-Option code)
- Easy Extensions for **extra-framework cations**, **semi-flexible frameworks** (like LEGOs!)
- Machine-Learning potentials for framework-adsorbate interactions
- supports both  **PyTorch** and [Tensorflow](#)
- **High-Throughput Calculation** (HTC) mode (experimental)
- Intel **OneAPI-SYCL** version of the code for portability on **Non-Nvidia** platforms (experimental)

1.2 🧑 Moves

I got the ~Moves~ like:

- Translation Move
- Rotation Move
- Reinsertion Move
- Swap (Insertion/Deletion) Move
- Identity-Swap moves for mixtures
- Gibbs Volume Change moves (For NVT-Gibbs)
- Gibbs Particle Transfer moves (For NVT-Gibbs)
- Semi-flexible framework Monte Carlo moves

1.3 🦴 Backbones & Algorithms

- Configuration-Bias Monte Carlo (CBMC)
- Used in swap move, reinsertion move, identity-swap move
- The use of CBMC for swap moves can be turned off
- Continuous-Fractional-Component (CFC) coupled with CBMC (CBCFC)
- Ewald summation of Coulombic interactions

2. Examples

Here are a few examples of using the gRASP code. Click on them to view more!

2.1 Basic GCMC simulation

- running a simple GCMC simulation in gRASP
-  [Detailed Explanation](#)
-  [Simulation Files](#)

2.2 NU-2000 Linker Rotation

- Rotate a part of your MOF, xylene adsorption
-  [Detailed Explanation](#)
-  [Simulation Files](#)

2.3 Energy Comparison: gRASP vs RASP-2

- van der Waals and Coulombic energies explained
-  [Detailed Explanation](#)

2.4 Mixture GCMC of CO₂/Methane

- Tutorial of Mixture GCMC (with **Identity Swap** moves)
-  [Detailed Explanation](#)
-  [Simulation Files](#)

2.5 ML potential for framework-adsorbate interactions

- better energy descriptions
-  [Detailed Explanation](#)
-  [Simulation Files](#)

3. Input File

-  NOTE: You need to successfully compile the code to continue, if you haven't done so, go [here](#).

3.1 Basics

- Look at the flowchart below for general ideas about the input files

```
classDiagram
class Restart{
    * text file
    * configuration of system
}

class Movies{
    * LAMMPS data file
}

class OUTPUT{
    * FILES and INFO GENERATED
}
class RUN{
    * the actual process of the simulation
}

class Adsorbate_definition{
    * Critical_Properties
    * Molecule_Size
    * Atom_Positions
    * Conectivity
}

class CIF{
    * P1 symmetry
    * System size
    * positions
    * partial_charge (optional)
}

class simulation_input{
    * Does your system have charge?
    * Run Steps or Cycles?
    * Run for how long?
    * Simulation Type?
}

class FORCEFIELD{
    * dictates general and special interactions
}

class SPACE{
    * spatial information of the system
}

FORCEFIELD <|-- force_field_def
FORCEFIELD <|-- force_field_mixing_rules_def
FORCEFIELD <|-- pseudo_atoms_def
simulation_input <|-- FORCEFIELD
SPACE <|-- CIF
SPACE <|-- Adsorbate_definition
simulation_input <|-- SPACE
RUN <|-- simulation_input

OUTPUT <|-- RUN

Movies <|-- OUTPUT
Restart <|-- OUTPUT

class force_field_def{
    * tail correction (optional)
}
class force_field_mixing_rules_def{
    * Lennard-Jones
}
class pseudo_atoms_def{
    * Mass
    * Charge
    * Element
    * Polarization
}
```

- There are **FIVE** files needed to run the simplist simulation

- a. [simulation.input](#)
- b. [force_field_mixing_rules.def](#)
- c. [pseudo_atoms.def](#)
- d. [Crystallographic Information File \(CIF\)](#)
- e. Adsorbate definition file, for example: [methane.def](#)

- There are also **OPTIONAL** files

- a. [force_field.def](#)
- b. [Machine Learning models](#)

3.2 simulation.input

- gRASPA takes a very similar format as RASPA-2.
- However, to ensure reproducibility and strictness, we have **additional requirements**.

- a. The code checks for a [list of required keywords](#)

- If a keyword is missing, the code will terminate
-

3.3 force_field_mixing_rules.def

3.4 pseudo_atoms.def

3.5 CIF File

4. Installation

Here we show how to install gRAPPA on Ubuntu 24.04 and latest NVIDIA products

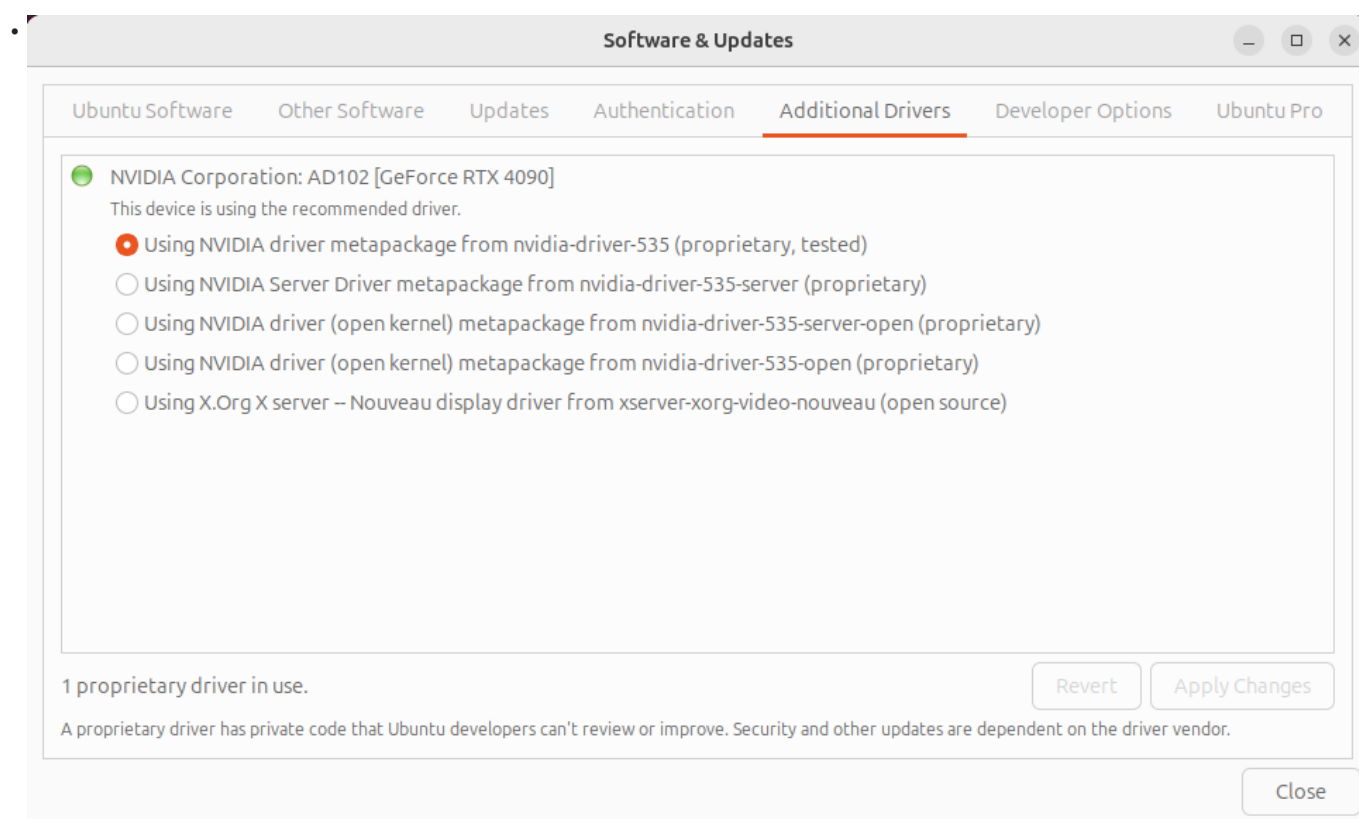
4.1 Prerequisites

- A machine with NVIDIA GPU
- Ubuntu 24.04 (clean version the best, but not required)

4.2 Steps (If you start with a clean Ubuntu 24.04)

4.2.1 1. Install NVIDIA Driver

- Find the recommended NVIDIA Driver
- 🖱️ Go to **Software & Updates**, select **Additional Drivers**
- select the **nvidia-drivers-535 (proprietary, tested)**
- click **Apply Changes**



4.2.2 2. Install NVIDIA CUDA Tool-kit

- open **terminal**
- Type:

```
sudo apt install nvidia-cuda-toolkit
```

4.2.3 3. Install NVHPC Compiler

- Go to the latest NVHPC compiler website
- Latest version is [24.5](#)
- Select a download method (tar file, for example)
- Follow their **instructions**
- 📄 Look at the final messages that the installer print to screen
- You will notice a line that adds nvhpc to PATH: `export PATH=/opt/nvidia/hpc_sdk/Linux_x86_64/24.5/compilers/bin:$PATH`
- **COPY THIS LINE!** to use for later

4.2.4 4. Download gRASPA

```
cd ~/
git clone https://github.com/snurr-group/gRASPA
```

4.2.5 5. Compile gRASPA

```
cd gRASPA/src_clean
cp ../NVC_COMPILE .
```

- Open **NVC_COMPILE**
- Add `export PATH=/opt/nvidia/hpc_sdk/Linux_x86_64/24.5/compilers/bin:$PATH` (the line you copied) to the top of **NVC_COMPILE**
- Add `-target=gpu` to **NVCFLAG** (easier option)
- if `-target=gpu` used, remove `-gpu=ccXX`
- if `-gpu=ccXX` used, you need to figure out `ccXX` for your GPU

```
chmod 777 NVC_COMPILE
./NVC_COMPILE
```

- 📄 Your COMPILATION FILE MAY LOOK LIKE THIS:

```
#!/bin/bash

rm nvc_main.x

export PATH=/opt/nvidia/hpc_sdk/Linux_x86_64/24.5/compilers/bin:$PATH

CXX="nvc++"

LINKFLAG="-L/usr/lib64/ -L/opt/local/lib/gcc11/"

NVCFLAG="-O3 -std=c++20 -target=gpu -fopenmp -cuda -stdpar=multicore"

$CXX $NVCFLAG $LINKFLAG -c axpy.cu
$CXX $NVCFLAG $LINKFLAG -c main.cpp
$CXX $NVCFLAG $LINKFLAG -c read_data.cpp
$CXX $NVCFLAG $LINKFLAG -c data_struct.cpp
$CXX $NVCFLAG $LINKFLAG -c VDW_Coulomb.cu
$CXX $NVCFLAG $LINKFLAG main.o read_data.o axpy.o data_struct.o VDW_Coulomb.o -o nvc_main.x

rm *.o
```

4.2.6 6. Run an Example (CO2 adsorption in MFI zeolite)

```
cd ../Examples/CO2-MFI/
../src_clean/nvc_main.x
```

4.2.7 7. That is it!!!

5. Installation

- Install on [Ubuntu](#)
- Install on [CentOS](#)
- Install on [Clusters](#)

6. Quick Start

Here we show the installation of gRASPA on clusters such as [Quest @ Northwestern](#) and on [Permuter @ LBNL](#)

Detailed installation process is documented [Install.md](#) for local machines such as for [Ubuntu 24.04](#) and [CentOS 7](#)

6.1 🤖 Installation

- 📝 NOTE: If you don't need deep potential, `cd src_clean/`, and start from [step 6](#).
- 📝 NOTE: Check for the **compute-capacity (cc)** for the GPU, it is related to the architecture of the GPU.

You need this to correctly compile the code! Here are some cc's for popular cards:

- Tesla A100: cc80
- RTX 30 series: cc86
- RTX 40 series: cc89
- 📝 use `-target=gpu` as an easy option in the compilation script

6.1.1 🛠️ Step 1

We download TensorFlow2 C++ API to a local directory: (assuming in the HOME directory)

```
mkdir tensorflow
cd tensorflow
wget https://storage.googleapis.com/tensorflow/libtensorflow/libtensorflow-gpu-linux-x86_64-2.11.0.tar.gz
tar -xvf libtensorflow-gpu-linux-x86_64-2.11.0.tar.gz
cd ..
vi .bashrc
```

6.1.2 🌟 Step 2

Add the following directories to environment variables in the `.bashrc` file:

```
export LIBRARY_PATH=$LIBRARY_PATH:~/tensorflow/lib
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:~/tensorflow/lib
```

6.1.3 🚀 Step 3

Then, we install [CppFlow](#):

```
git clone https://github.com/serizba/cppflow
cd cppflow
mkdir build
cd build
cmake -DCMAKE_PREFIX_PATH=~/tensorflow/ ..
make install DESTDIR=~/tensorflow/
```

6.1.4 🏀 Step 4

NERSC has its own PyTorch/LibTorch module, on Quest and other local machines, we recommend `libtorch-2.0.1-cu117`, please check for LibTorch @ pytorch.org

so now we can start patching gRASPA code with ML potential functionality. For example, if we want to use Allegro model which uses LibTorch:

```
mkdir patch_Allegro
```


6.1.5 🎵 Step 5

Modify line 64 in `patch.py` file to `patch_model=['Allegro']`. Then,

```
python patch.py
```

Now the patched source code is in `patch_Allegro/`. Go into the patched folder, and we have some final work to do.

```
cd patch_Allegro/
```

6.1.6 🍌 Step 6

Finally, we need to modify the source code due to NERSC configuration:

```
sed -i "s/std::filesystem/std::experimental::filesystem/g" *
sed -i "s/<filesystem>/<experimental\filesystem>/g" *
```

6.1.7 🦋 Step 7

Then, copy `NVC_COMPILE_NERSC` to the source code folder, and compile the code in the folder as follows:

- 📌 NOTE: If you use the vanilla version, simply replace `NVC_COMPILE_NERSC` with `NVC_COMPILE_NERSC_VANILLA` and proceed.

```
chmod +x NVC_COMPILE_NERSC
./NVC_COMPILE_NERSC
```

Remember to change `cppflowDir` and `tfDir` directories in `NVC_COMPILE_NERSC` if you install CppFlow and TensorFlow API in different directories. You might see some warning messages during compilation, just ignore them. Once ready, you will see a binary executable `nvc_main.x` in the folder.



GG~ Good work! Now you are ready to go!

7. Required Keywords

7.1 Here are the required keywords in

- These required input files can be divided into several sections, click to check them
- a. [simulation cycles and steps setup](#)
- b. [general simulation setup](#)
- c. [CBMC setup](#)
- d. [Force field setup](#)
- e. [Framework setup](#)
- f. [Adsorbate setup](#)

7.2 Cycles and Steps

7.2.1 **NumberOfInitializationCycles** (size_t)

- Number of Initialization cycles, the system gets initialized, the statistical averages are **NOT** gathered
- For CBCFC, the biasing factor η are **NOT** sampled nor applied
- For GC-TMMC, the biasing factor η are **NOT** sampled nor applied

7.2.2 **NumberOfEquilibrationCycles** (size_t)

- Number of Equilibration cycles, the statistical averaged are **NOT** gathered
- For CBCFC, the biasing factor η are sampled, and applied
- For GC-TMMC, the biasing factor η are sampled, and applied

7.2.3 **NumberOfEquilibrationCycles** (size_t)

- Number of Production cycles, the statistical averaged are gathered and printed to screen
- For CBCFC, sampling completes, the sampled biasing factor η are applied
- For GC-TMMC, the biasing factor η are sampled, and applied

7.2.4 **UseMaxStep** (bool)

- Whether to set a maximum number of steps per Monte Carlo cycle
- If `yes`, you need a following keyword `MaxStepPerCycle` (size_t)
- If `no`, then gRASPA uses the convention from RASPA-2: $\min(20, N_{\text{Mol}})$, where N_{Mol} is the number of adsorbate molecules in the simulation box

7.3 General Setup

7.3.1 **RestartFile** (bool)

- Whether to read system configuration (atom positions, forces, charges, scales, ...) from **RestartInitial** folder at the beginning of the simulation
- See [output section](#) for more information of the format of the **RestartInitial** folder

7.3.2 RandomSeed (size_t)

- Random seed for the series of random numbers generated during the simulation
- 📌 NOTE: This is recommended for reproducibility of results

7.3.3 AdsorbateAllocateSpace (size_t)

- Amount of space to pre-allocate before the simulation starts for each adsorbate component
- For example, if `AdsorbateAllocateSpace 10240`, then for **each** adsorbate component, it can store 10240 atoms on the GPU memory
- 🧠 TIP: you can kind-of estimate the max number of atoms your system can have. If you are unsure, use a large value, such as `10240`
- If your value is too small, and your insertion move goes beyond that, program will **terminate**

7.3.4 NumberOfSimulations (size_t)

- Number of simulation boxes the program will have
- For NVT-Gibbs, use `2`, otherwise, use `1`

7.3.5 SingleSimulation (bool)

- Whether to run only the selected simulation box (**by default simulation 0**)
- If NVT-Gibbs, then use `SingleSimulation no`, since the two boxes need to be simulated at the same time
- Other cases, use `SingleSimulation yes`
- Could have related (unimplemented) uses, if there are multiple independent boxes simulated at the same time

7.3.6 Temperature (double)

- temperature for the simulation, in the unit of **Kelvin**

7.3.7 Pressure (double)

- external pressure for the simulation box, in the unit of **Pascal**
- **NOT** used for NVT-Gibbs, just put in some value

7.4 🧪 CBMC Setup

7.4.1 NumberOfTrialPositions (size_t)

- Number of trial positions for a single (first) bead
- 🧠 TIP: `NumberOfTrialPositions 10` is recommended

7.4.2 NumberOfTrialOrientations (size_t)

- Number of trial orientation for different orientations of a rigid body
- 🧠 TIP: `NumberOfTrialOrientations 10` is recommended

7.5 ★ Force Field Setup

7.5.1 ChargeMethod (string)

- can only choose between `None` and `Ewald`
- Use `ChargeMethod None` if there is no partial charge on your adsorbate molecule, for example, alkane adsorption in rigid framework
- Use `ChargeMethod Ewald` for TIP4P water, carbon-dioxide molecules

7.5.2 OverlapCriteria (double)

- Rejection criteria if the **van der Waals (VDW)** interaction energy between a pair of atoms **from different molecules** exceed the input value here.
- You want the value to be big enough so that reasonable configurations are kept, but not that big so that you can save some computation time
- 🧠 TIP: `OverlapCriteria 1e5` is recommended for most cases

7.5.3 CutOffVDW (double)

- Cutoff for VDW interactions between a pair of atoms
- 🧠 TIP: `CutOffVDW 12.8` is a reasonable choice

7.5.4 CutOffCoulomb (double)

- Cutoff for short-range Coulombic interactions between a pair of atoms
- 🧠 TIP: `CutOffCoulomb 12.0` is a reasonable choice

7.5.5 EwaldPrecision (double)

- Precision of the Ewald summation, determines the number of k-points in x, y, z directions
- 🧠 TIP: `EwaldPrecision 1e-6` is a recommended

7.5.6 UseChargesFromCIFFile (bool)

- Dictates from where to read partial charges of the framework
- if `UseChargesFromCIFFile yes`, then read from CIF file, see [CIF file specifications](#)
- else, then the partial charges are from [pseudo_atoms.def](#)

7.6 🕸 Framework Setup

7.6.1 InputFileType (string)

- currently only `InputFileType CIF` is allowed, legacy versions uses `InputFileType POSCAR` (now disallowed)

7.6.2 FrameworkName (string)

- Name of the framework, if you use `IRMOF-1.cif`, then for this keyword, use `FrameworkName IRMOF-1`
- 📁 NOTE: each simulation box needs a "Framework", for NVT-Gibbs, just use a **CIF** file with no atoms but box lengths
- 🧠 TIP: For NVT-Gibbs, you could do `FrameworkName Empty-Box Empty-Box`
- see [NumberOfSimulations](#) keyword

7.6.3 UnitCells (size_t size_t size_t size_t)

- Requires **1** non-negative and **3** non-zero integers.
- a. the first one tells the code which framework it is, for example `Framework 0`,
- b. the latter 3 tells the code number of duplications to perform in x, y, z directions
- c. 📌 NOTE: each simulation box needs a "Framework", for NVT-Gibbs, just use a **CIF** file with no atoms but box lengths
- d. see **NumberOfSimulations** keyword
- e. 🧠 TIP: For NVT-Gibbs, you could do the following in `simulation.input`:

```
UnitCells 0 1 1 1
```

```
UnitCells 1 2 2 2
```

7.7 🔥 Adsorbate Setup

7.7.1 Component X MoleculeName (string)

- **X** is the Index of the adsorbate component, **starting from zero**
- If your adsorbate definition file is `c02.def`, and you want to do single component, then you do `Component 0 MoleculeName C02`
- For **mixture** adsorption, you can have multiple of these blocks, see [Mixture-Example]

7.7.2 IdealGasRosenbluthWeight (double)

- the **ideal-chain** rosenbluth weight of a molecule
- for chain molecule, in terms of n-alkane, starting from n-butane, are non-zero
- for rigid molecule and molecules with no dihedral terms, it is `1.0`

7.7.3 FugacityCoefficient (double)

- the fugacity coefficient applies directly to your value for `Pressure` to convert Pressure to **Fugacity**
- (Currently) can be calculated from RASPA-2 via Peng-Robinson Equation of State

7.7.4 CreateNumberOfMolecules (size_t)

- Number of molecules to create for each simulation box for the component

8. Chinese

8.1 主页

有关 gRASP 代码，请访问[我们的Github网页](#)。

8.1.1 🧠 功能

- 巨正则蒙特卡洛 (GCMC) 模拟小分子（例如水/CO₂/甲烷/二甲苯/.....）
- 巨正则系综中的**转移矩阵**蒙特卡洛 (TMMC) (GC-TMMC)
- 巨正则系综中的**混合物**吸附蒙特卡洛
- 正则吉布斯蒙特卡洛模拟（在快速选项代码中可用）
- 结构附带**阳离子**，**半柔性框架**（像积木乐高一样！）
- 用于结构-吸附质相互作用的机器学习势能
- 支持  PyTorch and  Tensorflow
- **高通量计算** (HTC) 模式（实验性）
- **OneAPI-SYCL** 版本代码以支持非 Nvidia平台（实验性）

8.1.2 🦋 蒙特卡洛移动

I got the ~Moves~ like (not Jagger):

- 平移
- 旋转
- 重新插入(Reinsertion)
- 交换（插入/删除
- 混合物的身份交换
- 吉布斯体积变化移动（用于正则吉布斯）
- 吉布斯粒子转移移动（用于正则吉布斯）
- 半柔性框架蒙特卡洛移动

8.1.3 🦋 内核和算法

- 结构偏置蒙特卡洛 (CBMC)
- 用于交换，重新插入，身份交换
- 交换可以关闭 CBMC
- 连续分数组分 (CFC) 与 CBMC 耦合 (CBCFC)
- 库仑相互作用的 Ewald 求和

8.2 示例

这里有几个使用 gRASPA 代码的例子。点击它们以查看更多内容！

8.2.1 基础 GCMC 模拟

- 在 gRASPA 中运行一个简单的GCMC模拟
-  [详细说明](#)
-  [模拟文件](#)

8.2.2 NU-2000 链接分子旋转

- 旋转部分MOF，对二甲苯吸附
-  [详细说明](#)
-  [模拟文件](#)

8.2.3 能量比较：gRASPA vs RASPA-2

- 范德华和库仑能量详解
-  [详细说明](#)

8.2.4 二氧化碳/甲烷混合物 GCMC

- 混合物 GCMC 教程（含身份交换动作）
-  [详细说明](#)
-  [模拟文件](#)

8.2.5 结构-吸附质相互作用的机器学习势

- 更好的结构-吸附质能量描述
-  [详细说明](#)
-  [模拟文件](#)

8.3 输入文件

- 🍷 你需要成功编译代码才能继续，如果你还没有完成，请[到这里](#)。

8.3.1 基础

查看下面的**流程图**以了解输入文件的一般概念

```
classDiagram
    class Restart{
        * 文本文件
        * 系统配置
    }

    class Movies{
        * LAMMPS 数据文件
    }

    class OUTPUT{
        * 生成的文件和信息
    }
    class RUN{
        * 实际的模拟过程
    }

    class Adsorbate_definition{
        * 临界属性
        * 分子尺寸
        * 原子位置
        * 连接性
    }

    class CIF{
        * P1 对称性
        * 系统尺寸
        * 位置
        * 部分电荷（可选）
    }

    class simulation_input{
        * 你的系统有电荷吗？
        * 运行步数或周期？
        * 运行多长时间？
        * 模拟类型？
    }

    class FORCEFIELD{
        * 指定一般和特殊的相互作用
    }

    class SPACE{
        * 系统的空间信息
    }

    FORCEFIELD <|-- force_field_def
    FORCEFIELD <|-- force_field_mixing_rules_def
    FORCEFIELD <|-- pseudo_atoms_def
    simulation_input <|-- FORCEFIELD
    SPACE <|-- CIF
    SPACE <|-- Adsorbate_definition
    simulation_input <|-- SPACE
    RUN <|-- simulation_input

    OUTPUT <|-- RUN

    Movies <|-- OUTPUT
    Restart <|-- OUTPUT

    class force_field_def{
        * 尾部修正（可选）
    }
    class force_field_mixing_rules_def{
        * Lennard-Jones
    }
    class pseudo_atoms_def{
        * 质量
        * 电荷
        * 元素
    }
```



```
* 极化  
}
```

- 运行最简单的模拟需要五个文件
- [simulation.input](#)
- [force_field_mixing_rules.def](#)
- [pseudo_atoms.def](#)
- 晶体学信息文件 (CIF)
- 吸附质定义文件，例如: methane.def
- 还有一些可选文件
- [force_field.def](#)
- [机器学习模型](#)

8.3.2 simulation.input

- **gRASP**A 的格式与 **RASP**A-2 非常相似。
- 但是，为了确保可重复性和严格性，我们有额外的要求。
- 代码会检查所需[关键字列表](#)
- 如果缺少[关键字](#)，代码将终止

8.3.3 force_field_mixing_rules.def

8.3.4 pseudo_atoms.def

8.3.5 CIF 文件

8.4 安装

- Install on [Ubuntu](#)
- Install on [CentOS](#)
- Install on [Clusters](#)

8.5 快速开始

这里我们展示了在集群上安装gRASPA的方法，例如在[Quest @ Northwestern](#)和[Permuter @ LBNL](#)上的安装方法

[详细的本地机器\(例如 Ubuntu 24.04\)的安装过程在这里](#)

8.5.1 🤖 安装

- 📌 NOTE: 如果你不需要深度潜能，cd src_clean/，从第 6 步开始。
- 📌 NOTE: 检查 GPU 的计算能力（cc），它与 GPU 的架构有关。

你需要这个来正确编译代码！ 以下是一些流行显卡的 cc：

- Tesla A100: cc80
- RTX 30 系列: cc86
- RTX 40 系列: cc89
- 📌 在编译脚本中使用 `-target=gpu` 作为简单选项

🔧 第一步

我们将 TensorFlow2 C++ API 下载到本地目录：（假设在 HOME 目录中）

```
mkdir ctensorflow
cd ctensorflow
wget https://storage.googleapis.com/tensorflow/libtensorflow/libtensorflow-gpu-linux-x86_64-2.11.0.tar.gz
tar -xvf libtensorflow-gpu-linux-x86_64-2.11.0.tar.gz
cd ..
vi .bashrc
```

🔧 第二步

将以下目录添加到 `.bashrc` 文件中的环境变量中：

```
export LIBRARY_PATH=$LIBRARY_PATH:~/ctensorflow/lib
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:~/ctensorflow/lib
```

🔧 第三步

然后，我们安装 CppFlow:

```
git clone https://github.com/serizba/cppflow
cd cppflow
mkdir build
cd build
cmake -DCMAKE_PREFIX_PATH=~/ctensorflow/ ..
make install DESTDIR=~/ctensorflow/
```

🏀 第四步

NERSC 有自己的 PyTorch/LibTorch 模块，在Quest和其他本地机器上，我们推荐 `libtorch-2.0.1-cu117`，请检查 PyTorch 的 [pytorch.org](#)

现在我们可以开始给 gRASPA 代码打补丁，以添加机器学习势功能。例如，如果我们想使用基于 LibTorch 的 Allegro 模型：

```
mkdir patch_Allegro
```

🎵 第五步

修改 `patch.py` 文件中的第 64 行为 `patch_model=['Allegro']`。然后，

```
python patch.py
```

现在打过补丁的源码在 `patch_Allegro/` 文件夹中。进入打过补丁的文件夹，我们还有一些最后的工作要做。

```
cd patch_Allegro/
```

🔥 第六步

最后，由于 NERSC 的配置，我们需要修改源码：

```
sed -i "s/std::filesystem/std::experimental::filesystem/g" *
sed -i "s/<filesystem>/<experimental\/filesystem>/g" *
```

🦋 第七步

然后，将 `NVC_COMPILE_NERSC` 复制到源码文件夹中，并按如下方式编译代码：

注意: 如果你使用原版，只需将 `NVC_COMPILE_NERSC` 替换为 `NVC_COMPILE_NERSC_VANILLA` 并继续。

```
chmod +x NVC_COMPILE_NERSC
./NVC_COMPILE_NERSC
```

如果你在不同目录中安装了 CppFlow 和 TensorFlow API，请记得更改 `NVC_COMPILE_NERSC` 中的 `cppflowDir` 和 `tfDir` 目录。编译过程中可能会看到一些警告信息，请忽略它们。一旦准备就绪，你将在文件夹中看到一个二进制可执行文件 `nvc_main.x`。



GG~ 干得好！现在你可以开始了！

8.6 Required Keywords

8.6.1 Here are the required keywords in [simulation.input](#)

- These required input files can be divided into several sections, click to check them

- [simulation cycles and steps setup](#)
- [general simulation setup](#)
- [CBMC setup](#)
- [Force field setup](#)
- [Framework setup](#)
- [Adsorbate setup](#)

8.6.2 🚲 Cycles and Steps

NumberOfInitializationCycles (size_t)

- Number of Initialization cycles, the system gets initialized, the statistical averages are **NOT** gathered
- For CBCFC, the biasing factor $\langle \eta \rangle$ are **NOT** sampled nor applied
- For GC-TMMC, the biasing factor $\langle \eta \rangle$ are **NOT** sampled nor applied

NumberOfEquilibrationCycles (size_t)

- Number of Equilibration cycles, the statistical averaged are **NOT** gathered
- For CBCFC, the biasing factor $\langle \eta \rangle$ are sampled, and applied
- For GC-TMMC, the biasing factor $\langle \eta \rangle$ are sampled, and applied

NumberOfEquilibrationCycles (size_t)

- Number of Production cycles, the statistical averaged are gathered and printed to screen
- For CBCFC, sampling completes, the sampled biasing factor $\langle \eta \rangle$ are applied
- For GC-TMMC, the biasing factor $\langle \eta \rangle$ are sampled, and applied

UseMaxStep (bool)

- Whether to set a maximum number of steps per Monte Carlo cycle
- If `yes`, you need a following keyword `MaxStepPerCycle` (size_t)
- If `no`, then gRASPA uses the convention from RASPA-2: $\langle \min(20, N_{\text{Mol}}) \rangle$, where $\langle N_{\text{Mol}} \rangle$ is the number of adsorbate molecules in the simulation box

8.6.3 🍷 General Setup

RestartFile (bool)

- Whether to read system configuration (atom positions, forces, charges, scales, ...) from **RestartInitial** folder at the beginning of the simulation
- See [output section](#) for more information of the format of the **RestartInitial** folder

RandomSeed (size_t)

- Random seed for the series of random numbers generated during the simulation

- 📌 NOTE: This is recommended for reproducibility of results

AdsorbateAllocateSpace (size_t)

- Amount of space to pre-allocate before the simulation starts for each adsorbate component
- For example, if `AdsorbateAllocateSpace 10240`, then for **each** adsorbate component, it can store 10240 atoms on the GPU memory
- 🧠 TIP: you can kind-of estimate the max number of atoms your system can have. If you are unsure, use a large value, such as `10240`
- If your value is too small, and your insertion move goes beyond that, program will **terminate**

NumberOfSimulations (size_t)

- Number of simulation boxes the program will have
- For NVT-Gibbs, use `2`, otherwise, use `1`

SingleSimulation (bool)

- Whether to run only the selected simulation box (**by default simulation 0**)
- If NVT-Gibbs, then use `SingleSimulation no`, since the two boxes need to be simulated at the same time
- Other cases, use `SingleSimulation yes`
- Could have related (unimplemented) uses, if there are multiple independent boxes simulated at the same time

Temperature (double)

- temperature for the simulation, in the unit of **Kelvin**

Pressure (double)

- external pressure for the simulation box, in the unit of **Pascal**
- **NOT** used for NVT-Gibbs, just put in some value

8.6.4 🧬 CBMC Setup

NumberOfTrialPositions (size_t)

- Number of trial positions for a single (first) bead
- 🧠 TIP: `NumberOfTrialPositions 10` is recommended

NumberOfTrialOrientations (size_t)

- Number of trial orientation for different orientations of a rigid body
- 🧠 TIP: `NumberOfTrialOrientations 10` is recommended

8.6.5 ★ Force Field Setup

ChargeMethod (string)

- can only choose between `None` and `Ewald`
- Use `ChargeMethod None` if there is no partial charge on your adsorbate molecule, for example, alkane adsorption in rigid framework
- Use `ChargeMethod Ewald` for TIP4P water, carbon-dioxide molecules

OverlapCriteria (double)

- Rejection criteria if the **van der Waals (VDW)** interaction energy between a pair of atoms **from different molecules** exceed the input value here.
- You want the value to be big enough so that reasonable configurations are kept, but not that big so that you can save some computation time
- 🧠 TIP: `OverlapCriteria 1e5` is recommended for most cases

CutOffVDW (double)

- Cutoff for VDW interactions between a pair of atoms
- 🧠 TIP: `CutOffVDW 12.8` is a reasonable choice

CutOffCoulomb (double)

- Cutoff for short-range Coulombic interactions between a pair of atoms
- 🧠 TIP: `CutOffCoulomb 12.0` is a reasonable choice

EwaldPrecision (double)

- Precision of the Ewald summation, determines the number of k-points in x, y, z directions
- 🧠 TIP: `EwaldPrecision 1e-6` is a recommended

UseChargesFromCIFFile (bool)

- Dictates from where to read partial charges of the framework
- if `UseChargesFromCIFFile yes`, then read from CIF file, see [CIF file specifications](#)
- else, then the partial charges are from [pseudo_atoms.def](#)

8.6.6 🕸 Framework Setup

InputFileType (string)

- currently only `InputFileType CIF` is allowed, legacy versions uses `InputFileType POSCAR` (now disallowed)

FrameworkName (string)

- Name of the framework, if you use `IRMOF-1.cif`, then for this keyword, use `FrameworkName IRMOF-1`
- 📄 NOTE: each simulation box needs a "Framework", for NVT-Gibbs, just use a **CIF** file with no atoms but box lengths
- 🧠 TIP: For NVT-Gibbs, you could do `FrameworkName Empty-Box Empty-Box`
- see [NumberOfSimulations](#) keyword

UnitCells (size_t size_t size_t size_t)

- Requires **1** non-negative and **3** non-zero integers.
- a. the first one tells the code which framework it is, for example `Framework 0`,
- b. the latter 3 tells the code number of duplications to perform in x, y, z directions
- c. 📄 NOTE: each simulation box needs a "Framework", for NVT-Gibbs, just use a **CIF** file with no atoms but box lengths
- d. see [NumberOfSimulations](#) keyword
- e. 🧠 TIP: For NVT-Gibbs, you could do the following in [simulation.input](#):

```
UnitCells 0 1 1 1
```

```
UnitCells 1 2 2 2
```

8.6.7 🔥 Adsorbate Setup

Component X MoleculeName (string)

- **X** is the Index of the adsorbate component, **starting from zero**
- If your adsorbate definition file is `CO2.def`, and you want to do single component, then you do `Component 0 MoleculeName CO2`
- For **mixture** adsorption, you can have multiple of these blocks, see [Mixture-Example]

IdealGasRosenbluthWeight (double)

- the **ideal-chain** rosenbluth weight of a molecule
- for chain molecule, in terms of n-alkane, starting from n-butane, are non-zero
- for rigid molecule and molecules with no dihedral terms, it is `1.0`

FugacityCoefficient (double)

- the fugacity coefficient applies directly to your value for `Pressure` to convert Pressure to **Fugacity**
- (Currently) can be calculated from RASPA-2 via Peng-Robinson Equation of State

CreateNumberOfMolecules (size_t)

- Number of molecules to create for each simulation box for the component

8.7 Examples

8.7.1 gRASP and RASPA-2 direct energy comparison!

介绍

- 人们总是想知道：我们可以依赖 gRASP 计算的能量吗？
- 🍷 我们可以（尝试）用 RASPA-2 确认！
- 我们将在 MOF 中进行 TIP4P 吸附！

准备

- **TIP4P.def**
- 这是分子定义文件
- **MOF.cif**
- 这是 CIF 格式的 MOF 定义文件（目前只允许 P1 对称性）
- **force_field_mixing_rules.def**
- **pseudo_atoms.def**
- **重启文件**
- 在 RASPA-2 中，根据 **结构、单元格、温度和压力** 命名重启文件（顺序很重要）。
- 像 **"RestartInitial/System_0/restart_MOF_4.2.2_298.000000_943"**
- 📄 RASPA-2 对于大于 100 万的数字使用科学记数法：用 "1e+06" 表示 100 万。
- 在 **gRASP** 中，只需命名为 **"restartfile"**
- 像 **"RestartInitial/System_0/restartfile"**

simulation.input

基本上，你读取重启文件，并运行 0 个周期/步数。

gRASPА

```
NumberOfInitializationCycles 0
NumberOfEquilibrationCycles 0
NumberOfProductionCycles 0
UseMaxStep yes
MaxStepPerCycle 1

RestartFile yes
RandomSeed 0

NumberOfTrialPositions 10
NumberOfTrialOrientations 10

NumberOfBlocks 1
AdsorbateAllocateSpace 10240
NumberOfSimulations 1
SingleSimulation yes

DifferentFrameworks no
InputFileType cif
FrameworkName qmof-71d1286
UnitCells 0 4 2 2

ChargeMethod Ewald
UseChargesFromCIFFile yes
Temperature 298
Pressure 943

OverlapCriteria 1e5
CutOffVDW 12.8
CutOffCoulomb 12.8
EwaldPrecision 1e-6

Component 0 MoleculeName TIP4P
IdealGasRosenbluthWeight 1.0
FugacityCoefficient 1.0
TranslationProbability 1.0
RotationProbability 1.0
ReinsertionProbability 1.0
SwapProbability 1.0
RunTMC no
UseBiasOnMacrostate no
TMMCMin 0
TMMCMax 10
CreateNumberOfMolecules 0
```

RASPA-2

```
SimulationType MonteCarlo
NumberOfCycles 0
NumberOfInitializationCycles 0

PrintEvery 1

RestartFile yes
ContinueAfterCrash no

Framework 0
FrameworkName qmof-71d1286
UnitCells 4 2 2
ExternalTemperature 298.0
ExternalPressure 943

Forcefield local
RemoveAtomNumberCodeFromLabel yes
UseChargesFromCIFFile yes
ChargeMethod Ewald
EwaldPrecision 1e-6
CutOffVDW 12.8
CutOffCoulomb 12.0

Component 0 MoleculeName TIP4P
StartingBead 0
MoleculeDefinition local
FugacityCoefficient 1.0
TranslationProbability 0.0
RotationProbability 0.0
ReinsertionProbability 0.0
SwapProbability 0.0
CreateNumberOfMolecules 0
```

结果比较

gRASPА

```
***** FINAL STAGE *****
VDW [Host-Host]: 0.00000 (0.00000 [K])
VDW [Host-Guest]: -105480.57019 (-199027.50030 [K])
VDW [Guest-Guest]: 235009.52229 (282651.05900 [K])
Real Coulomb [Host-Host]: 0.00000 (0.00000 [K])
Real Coulomb [Host-Guest]: -437987.31113 (-525695.52363 [K])
Real Coulomb [Guest-Guest]: -1373657.28204 (-1652130.97197 [K])
Ewald [Host-Host]: 589637.95832 (709171.89165 [K])
Ewald [Host-Guest]: -14747.79106 (-17737.52645 [K])
Ewald [Guest-Guest]: -41543.41156 (-49965.26996 [K])
DNN Energy: 0.00000 (0.00000 [K])
Tail Correction Energy: 0.00000 (0.00000 [K])
Total Energy: -1207868.88529 (-1452733.24107 [K])
```

RASPA-2

```
Current (full final energy) Energy Status
=====
Host/Adsorbate energy: -742474.08208755
Host/Adsorbate VDW energy: -199028.66021012
Host/Adsorbate Coulomb energy: -543445.42187744
Host/Adsorbate charge-charge Real energy: -520688.45228316
Host/Adsorbate charge-charge Fourier energy: -22756.96959428
Host/Adsorbate charge-bonddipole Real energy: 0.00000000
Host/Adsorbate charge-bonddipole Fourier energy: 0.00000000
Host/Adsorbate bonddipole-bonddipole Real energy: 0.00000000
Host/Adsorbate bonddipole-bonddipole Fourier energy: 0.00000000
...
Adsorbate/Adsorbate energy: -1419485.34470171
Adsorbate/Adsorbate VDW energy: 282651.85973471
Adsorbate/Adsorbate Coulomb energy: -1702137.20443643
Adsorbate/Adsorbate charge-charge Real energy: -1640567.82360744
Adsorbate/Adsorbate charge-charge Fourier energy: -61569.38082899
Adsorbate/Adsorbate charge-bonddipole Real energy: 0.00000000
Adsorbate/Adsorbate charge-bonddipole Fourier energy: 0.00000000
Adsorbate/Adsorbate bonddipole-bonddipole Real energy: 0.00000000
Adsorbate/Adsorbate bonddipole-bonddipole Fourier energy: 0.00000000
```

- 对于库仑相互作用的比较，直接比较 Real vs. Real 和 Fourier vs. Fourier 可能不一致
- 所以，比较 gRASPА 中的 **Real + Fourier** 与 RASPA-2 中的 **Real + Fourier**

能量	gRASPА	RASPA-2
Guest-Guest Real	-1652130.97	-1640567.82 ✗
Guest-Guest Fourier	-49965.27	-61569.38 ✗
Guest-Guest Real + Fourier	-1702096.24	-1702137.20 ✓

8.7.2 简单（也许吧）GCMC 模拟

介绍

- 沸石是结晶的微孔铝硅酸盐矿物，具有独特的三维蜂窝状结构，因其在各种工业和环境应用中的离子交换、吸附和催化性能而广泛应用。
- 在沸石中，MFI 在多孔材料社区中得到了广泛研究。

准备

- CO2.def**
- 这是分子定义文件
- MFI-2x2x2-P1.cif**
- 这是 MFI 沸石的 CIF 格式定义文件，使用 2x2x2 超单元（目前只允许 P1 对称性）
- force_field_mixing_rules.def**
- pseudo_atoms.def**

8.7.3 simulation.input

```
NumberOfInitializationCycles 5000
NumberOfEquilibrationCycles 0
NumberOfProductionCycles 0

UseMaxStep no
MaxStepPerCycle 1

NumberOfTrialPositions 10
NumberOfTrialOrientations 10

RestartFile no
RandomSeed 0
NumberOfBlocks 1
AdsorbateAllocateSpace 10240
NumberOfSimulations 1
SingleSimulation yes

InputFileType cif
FrameworkName MFI-2x2x2-P1
UnitCells 0 1 1 1
UseChargesFromCIFFile yes

ChargeMethod Ewald
Temperature 298
Pressure 1e4

OverlapCriteria 1e5
CutOffVDW 12.8
CutOffCoulomb 12.0
EwaldPrecision 1e-6

Component 0 MoleculeName CO2
          IdealGasRosenbluthWeight 1.0
          FugacityCoefficient 1.0
          TranslationProbability 1.0
          RotationProbability 1.0
          ReinsertionProbability 1.0
          SwapProbability 1.0
          CreateNumberOfMolecules 0
```

- 这个输入文件告诉 gRASP 在 298 K 和 10,000 帕的条件下运行 MFI 沸石中的 CO₂ 吸附 GCMC 模拟。

下面是一些详细说明

周期设置

```
NumberOfInitializationCycles 5000
NumberOfEquilibrationCycles 0
NumberOfProductionCycles 0
```

```
UseMaxStep no
MaxStepPerCycle 1
```

- 这些行告诉 gRASP 在初始化阶段运行 5000 个周期。
- 在初始化或平衡阶段，不计算系统平均值。只有在生产阶段才收集平均值。
- `UseMaxStep no` 告诉 gRASP 不要设置每个周期的最大步数
- 你可以通过设置 `UseMaxStep yes` 和 `MaxStepPerCycle 1` 来让 gRASP 以步数而不是周期运行

CBMC 设置

```
NumberOfTrialPositions 10
NumberOfTrialOrientations 10
```

- `NumberOfTrialPositions` 指定单个珠子的试验位置数（例如，第一个粒子）
- `NumberOfTrialOrientations` 是针对不同的方向（例如，在选择第一个粒子之后）

一般设置

```
RestartFile no
RandomSeed 0
NumberOfBlocks 1
AdsorbateAllocateSpace 10240
NumberOfSimulations 1
SingleSimulation yes
```

- `RestartFile no`: 是否使用重启文件，如果是 yes，请准备名为 `RestartInitial/System_0/restartfile` 的重启文件
- `RandomSeed 0` 设置随机数种子。最好是整数。
- `NumberOfBlocks` gRASP 默认使用 5 个块。
- `AdsorbateAllocateSpace 10240` 在 GPU 上为吸附质物种分配的插槽数。通过 10240，在模拟开始之前为每种吸附质物种分配 10240 个原子的空间。
- 分配一个大值，以便在每次移动时不必扩展此空间！
- 如果知道饱和负载，也可以节省一些空间。

盒子/框架设置

```
InputFileType cif
FrameworkName MFI-2x2x2-P1
UnitCells 0 1 1 1
UseChargesFromCIFFile yes
```

- `UnitCells 0 1 1 1`
- 第一个数：告诉 gRASP 这是哪个框架。
- 最后三个数：x、y 和 z 方向的单元格数量
- `UseChargesFromCIFFile yes`，使用 CIF 文件中提供的电荷。如果 `UseChargesFromCIFFile no`，电荷将从 `pseudo_atoms.def` 中获取。

力场设置

```
ChargeMethod Ewald
Temperature 298
Pressure 1e4

OverlapCriteria 1e5
CutOffVDW 12.8
CutOffCoulomb 12.0
EwaldPrecision 1e-6
```

* `OverlapCriteria 1e5` * 如果有一个对能量大于这个值（例如 1e5），这个试验位置/方向将被拒绝 * `EwaldPrecision 1e-6` * Ewald 求和的精度，值越大，精度越低。

```

Component 0 MoleculeName      CO2
            IdealGasRosenbluthWeight 1.0
            FugacityCoefficient    1.0
            TranslationProbability  1.0
            RotationProbability     1.0
            ReinsertionProbability  1.0
            SwapProbability         1.0
            CreateNumberOfMolecules 0

```

吸附质设置

- `IdealGasRosenbluthWeight 1.0` 链分子在特定温度下理想气体中的 Rosenbluth 权重
- 参见 [Macedonia and Maginn, 1998](#)
- `FugacityCoefficient 1.0` 分子的逸度系数 使用 Peng-Robinson EOS 自动计算逸度系数的功能, 要用 `FugacityCoefficient PR-EOS`。

8.7.4 机器学习势以更好地描述框架-吸附质能量：二氧化碳在 Mg-MOF74 中

介绍

使用机器学习势能（MLP），我们可以更好地描述 CO₂ 与 Mg-MOF74 的能量。

- 下面是使用 gRASPAs 中的 Allegro MLP 模型来描述框架-吸附质能量的例子。

simulation.input

DNN 基本信息

下面的块表示 MLP 的基本信息

```
UseDNNForHostGuest yes
DNNModelName co2-mof74-deployed.pth
DNNMethod Allegro
DNNEnergyUnit eV
MaxDNNDrift 100000.0
```

- `DNNMethod` 可以选择 `Allegro` 或 `LCLin`
- `MaxDNNDrift`
- 这个关键字表示经典能量和 MLP 能量之间可接受的最大能量差
- gRASPAs 如何处理这个问题：首先计算经典力场能量，然后将 DNN 能量作为“修正”计算
- 还要考虑一起调整 `OverlapCriteria` 这个关键字
- `OverlapCriteria` 控制是否因为一对原子之间的高 vdW 能量而被拒绝

组件信息

```
Component 0 MoleculeName      CO2
             IdealGasRosenbluthWeight 1.0
             FugacityCoefficient  1.0
             TranslationProbability 1.0
             RotationProbability  1.0
             ReinsertionProbability 1.0
             SwapProbability       1.0
             DNNPseudoAtoms C_co2 O_co2
             CreateNumberOfMolecules 0
```

- 🚩 **特别注意事项！**
- `DNNPseudoAtoms` 是必需的。
- 这表示在经典模拟中要在 MLP 中考虑的伪原子。
- 以 TIP4P 水为例。虚构的电荷位置不应被考虑。
- 请参阅此处的示例，了解[虚构 CO₂ 模型中的虚构原子位置](#)

8.7.5 混合物GCMC：二氧化碳和甲烷在 MOF MATVAF 中的吸附

介绍

- Bae 和共同作者发表了一篇使用混合配体MOF模拟 二氧化碳/甲烷分离的[论文](#)
- 他们使用 [MuSiC 代码](#)进行了混合物 GCMC 模拟

simulation.input

```
NumberOfInitializationCycles 100000
NumberOfEquilibrationCycles 0
NumberOfProductionCycles 100000

UseChargesFromCIFFile yes

UseMaxStep yes
MaxStepPerCycle 1

RestartFile no
RandomSeed 0

NumberOfTrialPositions 10
NumberOfTrialOrientations 10

NumberOfBlocks 1
AdsorbateAllocateSpace 10240
NumberOfSimulations 1
SingleSimulation yes

InputFileType cif
FrameworkName MATVAF_clean-Bae-PartialCharge-fixed-Original-N2-Charge
UnitCells 0 3 3 2

ChargeMethod Ewald
Temperature 296
Pressure 500000

OverlapCriteria 1e5
CutOffVDW 12.8
CutOffCoulomb 12.0
EwaldPrecision 1e-6

Component 0 MoleculeName C02
      IdealGasRosenbluthWeight 1.0
      FugacityCoefficient 0.9730021872
      MolFraction 0.5
      TranslationProbability 1.0
      RotationProbability 1.0
      ReinsertionProbability 1.0
      IdentityChangeProbability 1.0
      SwapProbability 1.0
      CreateNumberOfMolecules 0

Component 1 MoleculeName methane
      IdealGasRosenbluthWeight 1.0
      FugacityCoefficient 0.9894689168
      MolFraction 0.5
      TranslationProbability 1.0
      ReinsertionProbability 1.0
      IdentityChangeProbability 1.0
      SwapProbability 1.0
      CreateNumberOfMolecules 0
```

特别注意事项！

- gRASPA 支持使用 Peng-Robinson 状态方程计算逸度系数
- 对每个成分使用关键字 `FugacityCoefficient PR-EOS` 调用 Peng-Robinson 系数计算
- 使用 `IdentityChangeProbability 1.0` 改善采样
- 不使用身份交换也可以，因为 GCMC 交换移动就足够了。
- Bae 等人报告的值是 **过量负载**，而 gRASPA 目前报告的是 **绝对负载**
- 💡 你可以尝试将 gRASPA 重启文件输入到 RASPA-2 并运行 **零周期**
- 这将计算 gRASPA 重启文件中构型的过量负载。查看 [基础GCMC](#) 基础以了解更多

8.7.6 框架半柔性：pX 在 NU-2000 中

介绍

- **NU-2000** 是一种具有 3D-链接分子的 MOF，对对二甲苯具有高度选择性。
- 这里我们展示了一个使用半柔性框架表示的例子，即**链接分子旋转移动**，用于对二甲苯的吸附，并探索当链接分子可以旋转时，对二甲苯如何在 NU-2000 的通道内堆积。
- 链接分子可以围绕其轴旋转，旋转是自由的，意味着它可以旋转到任何随机角度。
- 链接分子可以在链接定义文件中进行处理。

simulation.input

框架定义

- `NumberOfFrameworkComponents <size_t>`
- 这个关键字将决定框架组件的数量
- 默认情况下，这个数量是 1
- 如果多于 1，一个组分将从框架中分离出来
- 框架 CIF 文件不必是超晶胞，但必须是 **P1 对称性**
- `Framework_Component_ <size_t>`
- 在这里提供分离框架组件的定义
- 你可以为该组件中的分子提供 MC 移动的概率
- 组件的定义需要在其他地方提供，详细解释见 framework-component-file
- 🗨 为了将这部分与吸附质组件区分开来，我们在关键字后面加一个下划线 "_"
- 在我们的例子中，这部分如下所示📄:

```
FrameworkName Al-Bicyclo-AddedH-P1-fix-PBC
SeparateFrameworkComponents yes
  NumberOfFrameworkComponents 2
UnitCells 0 4 2 2

Framework_Component_ 1
  TranslationProbability 0.0
  RotationSpecialProbability 1.0
END_OF_Framework_Component_ 1
```

框架组件文件

- 在 simulation.input 文件中定义的 CIF 文件中提供框架组件的原子索引
- 下面的示例名为 `Framework_Component_1.def`
- 名称必须与 simulation.input 文件中提到的组件匹配。
- 🗨 确保你的框架分子不跨越周期性边界（确保它们没有被边界切割）
- 示例如下所示📄:

```
#注意：此处编号应与 CIF 文件中的编号匹配。需要 P1 对称性。
#你不应该在这里定义的列表中制作超单元时有任何问题，数字将相应复制。
#分子索引应与原子索引匹配，还要确保你的分子不跨越 pbc
Framework_Component_Name: BiCyclo-Linker
Number_of_Molecules_for_Framework_Component: 4
Number_of_atoms_for_each_molecule: 22
Atom_Indices_for_Molecule 0: 5 6 7 8 9 10 11 12 13 14 15 16 17 18 20 21 22 23 25 26 27 28
Atom_Indices_for_Molecule 1: 38 39 40 41 42 43 44 45 46 47 48 49 50 51 53 54 55 56 58 59 60 61
Atom_Indices_for_Molecule 2: 67 68 69 70 71 72 73 74 75 76 77 78 79 80 82 83 84 85 86 87 88 89
Atom_Indices_for_Molecule 3: 93 94 95 96 97 98 99 100 101 102 103 104 105 106 108 109 110 111 112 113 114 115
```


其他用途：框架外阳离子/阴离子

- 类似的设置可以用于模拟框架外的阳离子/阴离子
- 例如，我们有一个带有 `-12 e` 的阴离子框架
- 如果框架外的阳离子是钙，我们需要每个单元中有 6 个 Ca^{2+}
- 这里是步骤：

a. 我们首先在 CIF 文件中添加 6 个 Ca^{2+} ，如下所示

-  阳离子不必在完美的位置，只要不产生重叠。

```
Ca 1 0.5 0.5 0.5 Biso 1 Ca 1
Ca 1 0.55 0.5 0.5 Biso 1 Ca 1
Ca 1 0.6 0.55 0.5 Biso 1 Ca 1
Ca 1 0.65 0.45 0.5 Biso 1 Ca 1
Ca 1 0.45 0.65 0.5 Biso 1 Ca 1
Ca 1 0.45 0.6 0.55 Biso 1 Ca 1
```

b. 在 `Framework_Component_1.def` 中添加这些阳离子分子的定义

```
#注意：此处编号应与 CIF 文件中的编号匹配。需要 P1 对称性。
#你不应该在这里定义的列表中制作超单元时有任何问题，数字将相应复制。
#分子索引应与原子索引匹配，还要确保你的分子不跨越 pbc
Framework_Component_Name: Ca_Ion
Number_of_Molecules_for_Framework_component: 6
Number_of_atoms_for_each_molecule: 1
Atom_Indices_for_Molecule 0: 306
Atom_Indices_for_Molecule 1: 307
Atom_Indices_for_Molecule 2: 308
Atom_Indices_for_Molecule 3: 309
Atom_Indices_for_Molecule 4: 310
Atom_Indices_for_Molecule 5: 311
```

3. 在 `simulation.input` 文件中，添加以下几行

-  这两个代码块将告诉 gRASPA 将 CIF 文件中的原子分离为两个（框架 + 阳离子）宿主组件，并将平移移动分配给阳离子。

```
FrameworkName xxx
SeparateFrameworkComponents yes
NumberofFrameworkComponents 2
```

和

```
Framework_Component_1
  TranslationProbability 1.0
END_OF_Framework_Component_1
```